Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 459

Name: Vibhu Shihora

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [7]:
```
class Rectangle :

    def area(self,side,side2):
        return side * side2

r = Rectangle()

print(r.area(5,6))
```

```
30
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [4]:
```
class Square :

    def __init__(self,side):
        self.side = side

    def area(self):
        return self.side * self.side

    def output(self):
```

```
        return "Area : " + str(self.area())

r = Square(10)


print(r.output())
# print(r.side)
```

```
Area : 100
10
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [12]:
```
class ItsNotRectangle(Exception):
    pass

class Rectangle :

    def __init__(self,side,side2):
        self.side , self.side2 = self.compareSide(side,side2)

    def area(self):
        return self.side * self.side2

    def output(self):
        return "Area : " + str(self.area())

    @classmethod
    def compareSide(cls, side1, side2):
        if side1 == side2:
            raise ItsNotRectangle("THIS IS SQUARE")
        return side1, side2

try:
    r = Rectangle(10,20)
except Exception as msg:
    print(msg)
else:
    print(r.output())
```

```
Area : 200
```

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [7]:
```
class Square :

    def __init__(self,side):
        self.__side = side

    def area(self):
        return self.__side * self.__side

    def setSide(self,newSide):
        self.__side = newSide

    def getSide(self):
        return self.__side

    def output(self):
        return "Area : " + str(self.area())

r = Square(10)

# print(r.output())

# print(r.getSide())  # Get

r.setSide(20)  # Set

print(r.getSide())  # Get

# print(r.__side) #We Cannot Access Private Member Directly
```

```
20
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two methods getBalance() and printBalance().

In [12]:
```
class Profit:

    def getProfit(self):
         self.profit = float(input("Enter Profit: "))

class Loss:

    def getLoss(self):
        self.loss = float(input("Enter Loss: "))

class BalanceSheet(Profit,Loss):

    def getBalance(self):
        return self.profit - self.loss

    def printBalance(self):
```

```
        print("Balance Is : " + str(self.getBalance()))

b = BalanceSheet()
b.getProfit()
b.getLoss()
b.getBalance()
b.printBalance()
```

```
Balance Is : 79.0
```

15) WAP to demonstrate all types of inheritance.

In [14]:

```
# 1.Single Level

class Parent:
    def show(self):
        print("This is Parent class")


class Child(Parent):
    def display(self):
        print("This is Child class")


c = Child()
c.show()
c.display()

# 2.Multiple

class Father:
    def showFather(self):
        print("Father class")


class Mother:
    def showMother(self):
        print("Mother class")


class Child(Father, Mother):
    def showChild(self):
        print("Child class")


c = Child()
c.showFather()
c.showMother()
c.showChild()

# 3.Multilevel

class Grandparent:
    def gp(self):
        print("Grandparent class")


class Parent(Grandparent):
    def p(self):
```

```
        print("Parent class")


class Child(Parent):
    def c(self):
        print("Child class")


obj = Child()
obj.gp()
obj.p()
obj.c()

# 4.Hierarchical

class Parent:
    def show(self):
        print("Parent class")


class Child1(Parent):
    def c1(self):
        print("Child1 class")


class Child2(Parent):
    def c2(self):
        print("Child2 class")


obj1 = Child1()
obj2 = Child2()

obj1.show()
obj1.c1()

obj2.show()
obj2.c2()

# 5.Hybrid

class A:
    def methodA(self):
        print("Class A")


class B(A):
    def methodB(self):
        print("Class B")


class C(A):
    def methodC(self):
        print("Class C")


class D(B, C):
    def methodD(self):
        print("Class D")
```

```
obj = D()
obj.methodA()
obj.methodB()
obj.methodC()
obj.methodD()

# ama java ni  jem if apde child no constructor banvi e to parent no constructor
# super().__init__() child na constructor ma

# Yes, in Python you can call super().__init__() inside any method
# In Java, you cannot call a constructor from a normal method

# class Parent:
#     def __init__(self):
#         print("Parent constructor")


# class Child(Parent):
#     pass


# c = Child()

# ama If child does NOT define constructor, then parent is called automatically
```

```
This is Parent class
This is Child class
Father class
Mother class
Child class
Grandparent class
Parent class
Child class
Parent class
Child1 class
Parent class
Child2 class
Class A
Class B
Class C
Class D
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [18]:
```
class Person:

    def __init__(self,name,age):
        self.name = name
```

```
        self.age = age

class Employee(Person):

    def __init__(self,name,age,salary):
        super().__init__(name,age)
        self.salary = salary

    def __str__(self):
        return "Name : " + self.name + "\nAge : " + str(self.age) + "\nSalary :

e = Employee("vi",19,99999999)
print(e)
```

```
Name : vi
Age : 19
Salary : 99999999
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [20]:
```
from abc import ABC, abstractmethod

# ABC etle Abstract Base Class

class Shape(ABC):

    @abstractmethod
    def draw(self):
        pass

class Rectangle(Shape):
    def draw(self):
        print("Rectangle")

class Circle(Shape):
    def draw(self):
        print("Circle")

class Triangle(Shape):
    def draw(self):
        print("Triangle")

s = []
s.append(Rectangle())
s.append(Circle())
s.append(Triangle())
```

```
for i in s:
    i.draw()
```

```
Rectangle
Circle
Triangle
```